What are the decidable and undecidable problems in TOC?

A problem is called decidable, when there is a solution to that problem and also can construct algorithms corresponding to that. For example, find all the odd numbers in the range from 1 to 50. For this problem, we can easily find a solution by constructing an algorithm.

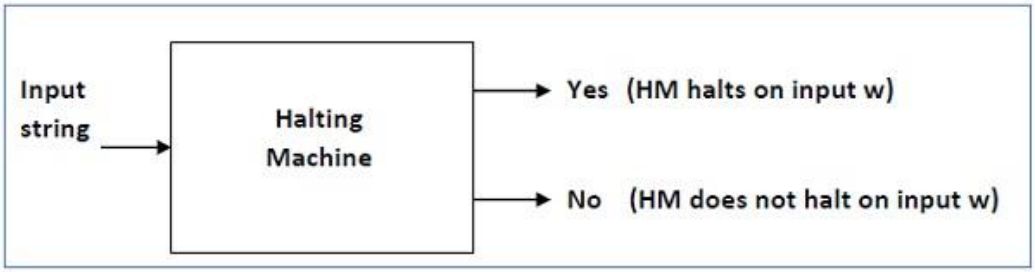
In terms of Turing Machine (TM), if a problem is decidable, then the Turing machine halts whether or not it accepts its input.

The problems for which we can't construct an algorithm that can answer the problem correctly in the infinite time are termed as Undecidable Problems in the theory of computation (TOC).

A problem is undecidable if there is no Turing machine that will always halt an infinite amount of time to answer as 'yes' or 'no'.

Turing Machine Halting Problem

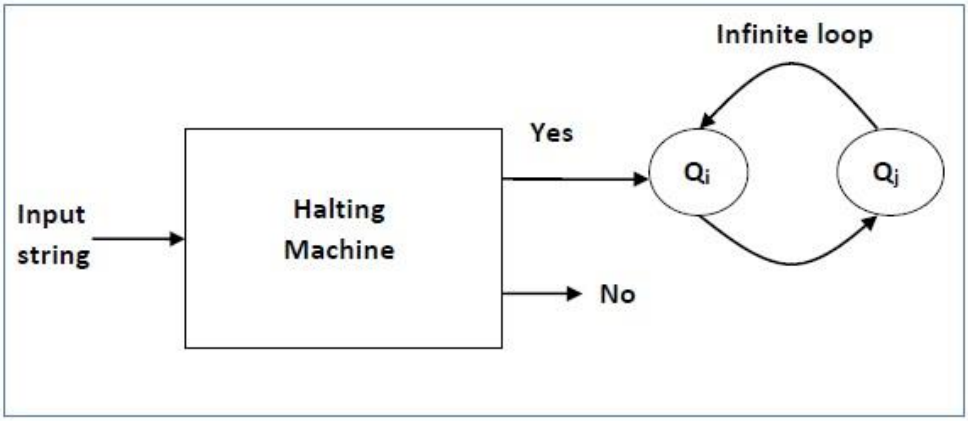
Input − A Turing machine and an input string **w**.
Problem − Does the Turing machine finish computing of the string **w** in a finite number of steps? The answer must be either yes or no.
Proof − At first, we will assume that such a Turing machine exists to solve this problem and then we will show it is contradicting itself. We will call this Turing machine as a **Halting machine** that produces a 'yes' or 'no' in a finite amount of time. If the halting machine finishes in a finite amount of time, the output comes as 'yes', otherwise as 'no'. The following is the block diagram of a Halting machine −



Now we will design an **inverted halting machine (HM)'** as −

- If **H** returns YES, then loop forever.
- If **H** returns NO, then halt.

The following is the block diagram of an 'Inverted halting machine' −



Further, a machine **$(HM)_2$** which input itself is constructed as follows −

- If $(HM)_2$ halts on input, loop forever.
- Else, halt.

Here, we have got a contradiction. Hence, the halting problem is **undecidable**.